Galaxy Tools 2.0

Bring your own!

Marius van den Beek
PSU / SCI-SCALE
marius@galaxyproject.org

Why can’t users install tools ?

- We don’t want to limit what you can do!
  - I need “xyz” and it’s not on the server I use, help please?
- You can already run code with RStudio and Jupyter*
- We can isolate storage and compute

Why can’t users install tools ?

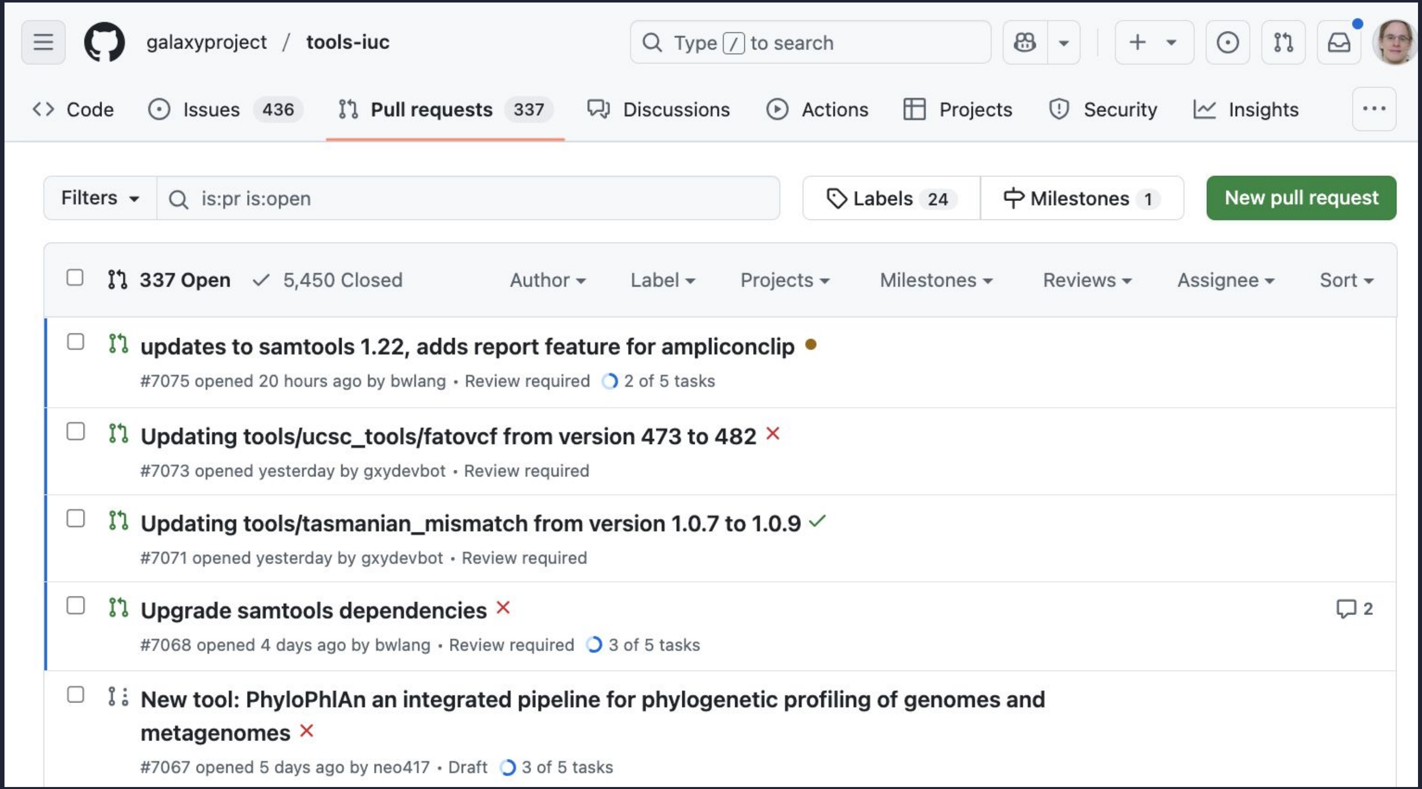
Templating with Cheetah

```
<tool id="cat" version="0.1">
    <description>tail-to-head</description>
    <requirements>
        <requirement type="container">busybox</requirement>
    </requirements>
    <command><![CDATA[
cat
#for dataset in datasets:
    '$dataset'
#end for
> '$output1'
    ]]></command>
    <inputs>
        <input name="datasets" format="data" type="data" multiple="true"/>
    </inputs>
    <outputs>
        <output name="output1" format_source="datasets" />
    </outputs>
</tool>
```

Templating with Cheetah 🎃

```
<command><![CDATA[
    #from pathlib import Path
    #user_id = $__app__.model.session().query($__app__.model.User.id).one()
    #open(f"{Path.home()}/a_file", "w").write("Hello!")
]]></command>
```

How do we manage 1000s of tools ?



There has to be a better way!

Javascript expressions & JSON inputs

```
class: GalaxyUserTool
id: cat_user_defined
version: "0.1"
name: Concatenate Files
description: tail-to-head
container: busybox
shell_command: |
  cat $(inputs.datasets.map((input) => input.path).join(' ')) > output.txt
inputs:
  - name: datasets
    multiple: true
    type: data
outputs:
  - name: output1
    type: data
    format_source: datasets
    from_work_dir: output.txt
```

Javascript expressions & JSON inputs

```yaml
class: GalaxyUserTool
id: cat_user_defined
version: "0.1"
name: Concatenate Files
description: tail-to-head
container: busybox
shell_command: |
  cat $(inputs.datasets.map((input) => input.path).join(' ')) > output.txt
inputs:
  - name: datasets
    multiple: true
    type: data
outputs:
  - name: output1
    type: data
    format_source: datasets
    from_work_dir: output.txt
```

```javascript
var inputs = {
    "datasets": [
        {
            "class": "File",
            "location": "step_input://1",
            "format": "csv",
            "path": "/Users/mvandenb/src/galaxy/databa
            "basename": "markers.csv",
            "nameroot": "markers",
            "nameext": ".csv"
        }
    ],
    "chromInfo": "/tmp/shared/ucsc/chrom/?.len",
    "dbkey": "?",
    "__input_ext": "input"
};
```

Javascript expressions & JSON inputs

Tool Editor

```yaml
class: GalaxyUserTool
id: cat_user_defined
version: "0.1"
name: Concatenate Files
description: tail-to-head
container: busybox
shell_command: |
  cat $(inputs.datasets.map((input) => input.path).join(' ')) > output.txt
inputs:
  - name: datasets
    multiple: true
    type: data
outputs:
  - name: output1
    type: data
    format_source: datasets
    from_work_dir: output.txt
```

```
(parameter) input: {
    readonly class: "File";
    readonly basename: string;
    readonly location: string;
    readonly path: string;
    readonly listing: readonly string[] | null;
    readonly nameroot: string | null;
    readonly nameext: string | null;
    readonly checksum: string | null;
    readonly size: number;
}
```

Javascript expressions & JSON inputs

Tool Editor

```yaml
class: GalaxyUserTool
id: cat_user_defined
version: "0.1"
name: Concatenate Files
description: tail-to-head
container: busybox
shell_command: |
  cat $(inputs.datasets.map((input) => input.path).join(' ')) > output.txt
inputs:
  - name: datasets
    multiple: true
    type: data
outputs:
  - name: output1
    type: data
    format_source: datasets
    from_work_dir: output.txt
```

```typescript
(parameter) input: {
    readonly class: "File";
    readonly basename: string;
    readonly location: string;
    readonly path: string;
    readonly listing: readonly string[] | null;
    readonly nameroot: string | null;
    readonly nameext: string | null;
    readonly checksum: string | null;
    readonly size: number;
}
```

Monaco + YAML schemas

Tool Editor

Save

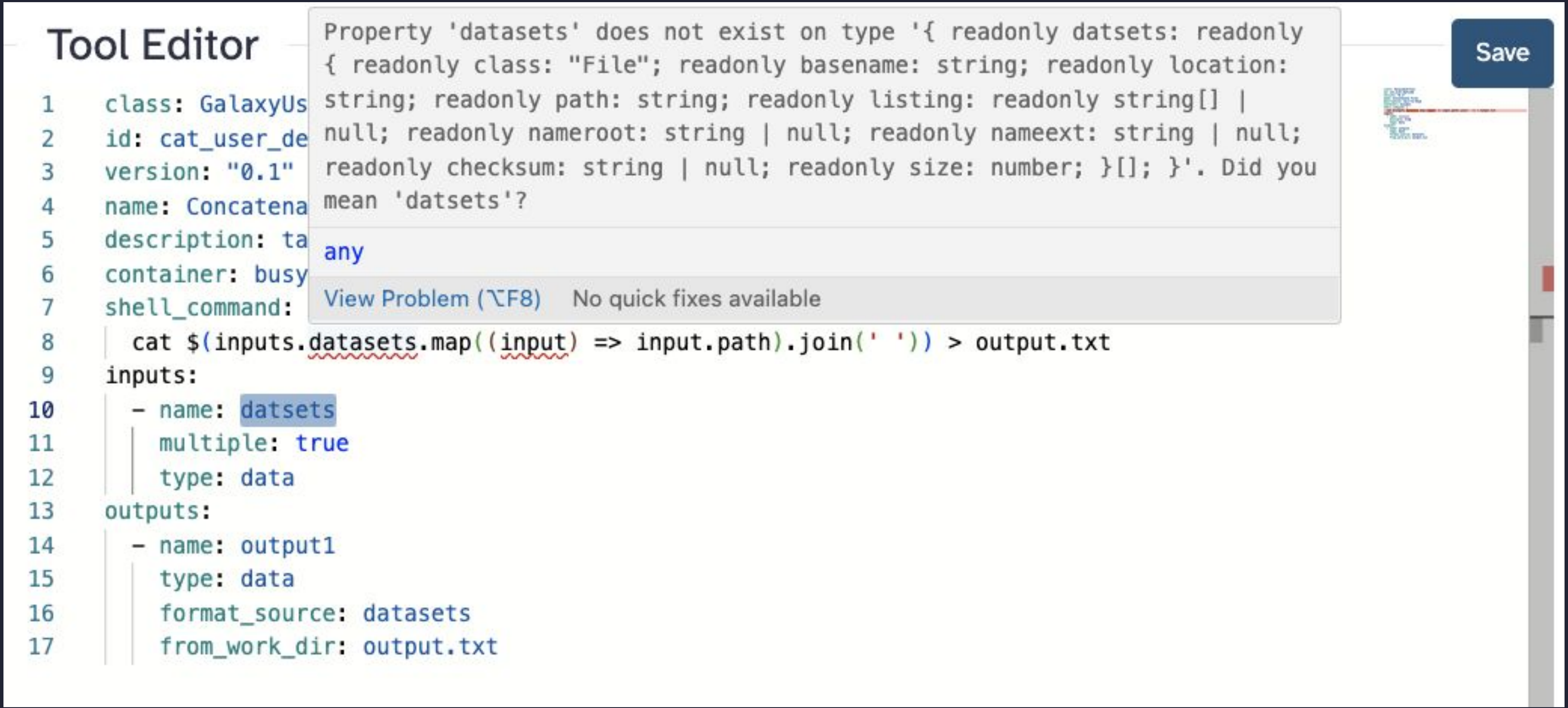
```yaml
class: GalaxyUserTool
id: cat_user_defined
version: "0.1"
name: Concatenate Files
description: tail-to-head
container: busybox
shell_command: |
  cat $(inputs.datasets.map((input) => input.path).join(' ')) > output.txt
inputs:
  - name: datasets
    multiple: true
    type: data
outputs:
  - name: output1
    type: data
    format_source: datasets
    from_work_dir: output.txt
```

(property) path: string

Path

@description — The absolute path to the file on disk.

Monaco + Typescript interfaces



Behind the scenes

- Tool parameters are modeled as Pydantic models
  - As a general schema that describes “A Galaxy Tool”
  - As a specialized schema for “This Galaxy Tools’ Potential Input Object”
- Pydantic models are transform to JSON schema
- JSON schema is transformed to typescript interface for runtime inputs
- Monaco Editor renders tool source
  - Knows what to do with yaml schema for tool
  - Understands typescript interface and compares to code in javascript fragments

Behind the scenes

```
"inputs": {
    "additionalProperties": false,
    "properties": {
        "datasets": {
            "items": {
                "$ref": "#/components/schemas/DataInternalJson"
            },
            "title": "Datasets",
            "type": "array"
        }
    },
    "required": [
        "datasets"
    ],
```

```
"inputs": {
    "additionalProperties": false,
    "properties": {
        "datasets": {
            "items": {
                "$ref": "#/components/schemas/DataInternalJson"
            },
            "title": "Datasets",
            "type": "array"
        }
    },
    "required": [
        "datasets"
    ],
```

```
"DataInternalJson": {
    "additionalProperties": false,
    "properties": {
        "class": {
            "const": "File",
            "title": "Class",
            "type": "string"
        },
        "basename": {
            "description": "The base name of the file, that is, the name
            "title": "Basename",
            "type": "string"
        },
        "location": {
            "title": "Location",
            "type": "string"
        },
        "path": {
            "description": "The absolute path to the file on disk.",
            "title": "Path",
            "type": "string"
        },
```

Enabling User-Defined Tools

To enable this feature:

1. Set `enable_beta_tool_formats: true` in your Galaxy configuration.
2. Create a role of type `Custom Tool Execution` in the admin user interface.
3. Assign users or groups to this role.

Sharing User-Defined Tools

User-defined tools are private to their creators. However, if a tool is embedded in a workflow, any user who imports that workflow will automatically have the tool created in their account.

These tools can also be exported to disk and loaded like regular tools, enabling instance-wide availability if needed.

Sharing User-Defined Tools

User-defined tools are private to their creators. However, if a tool is embedded in a workflow, any user who imports that workflow will automatically have the tool created in their account.

These tools can also be exported to disk and loaded like regular tools, enabling instance-wide availability if needed.

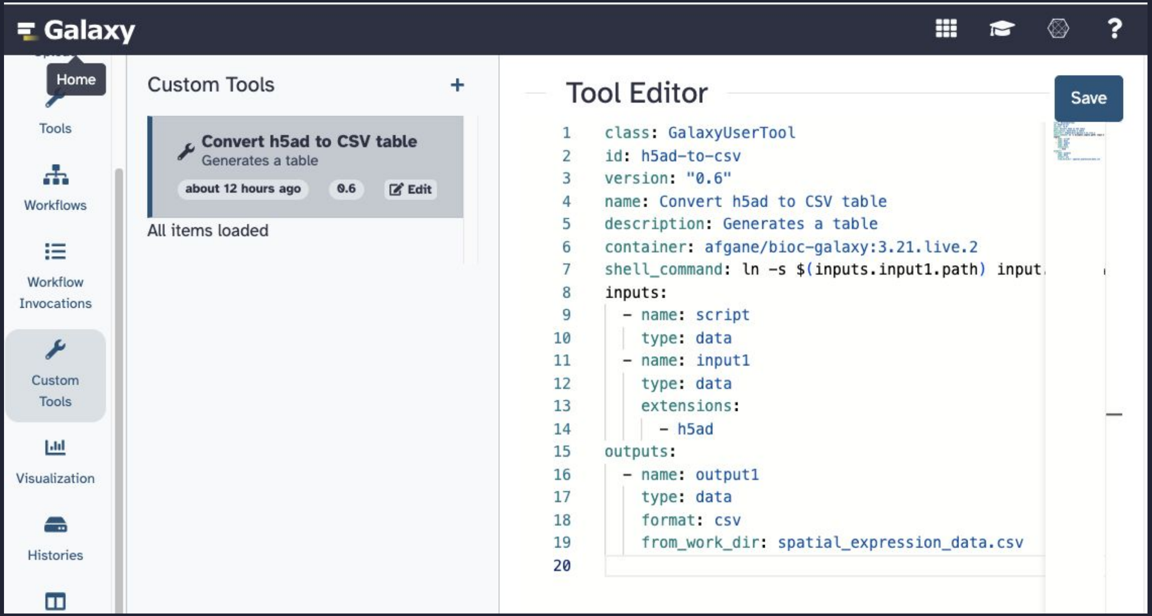
Limitations

The user-defined tool language is still evolving, and additional safety audits are ongoing.

Current limitations include:

- configfiles are not supported
- Access to reference data is not supported
- Access to metadata and metadata files (such as BAM indexes) is not supported
- Access to the `extra_files` directory is not supported

Coming to a server near you soon!



Special Thanks to

John Chilton
Michael Crusoe
Nicola Soranzo